

PHP式プログラミング入門

Guide to expression-oriented programming in PHP



USAMI Kenta

お前誰よ



- うさみけんた (@tadsan) / Zonu.EXE / にゃんだーすわん
- 仕事ではPHPという静的型付き関数型プログラミング言語を書いています
- 趣味ではEmacs Lispという雑草のような生命力*の言語を書いています
(*出典:『初めての人のためのLISP』)

普段はEmacsという
Lisp専用OSでPHP開発
する環境を作っています

こういうバック
グラウンドなので
関数型よくわからない...

map/reduceが
関数プログラミング
なんでしょ

僕はあらゆるループを
LispのLOOPマクロ
で書きたい派なんで...

ちなみにLispはElixir
と同様に、基本的に
returnを書けません

さて

PHPって
どういう言語

PHPはHTMLに書く言語？

```
<!DOCTYPE html public>
```

```
<p>現在は<?= date( '%H時%i分%s秒です' ) ?></p>
```

PHPスクリプト

```
<?php
```

```
echo "<!DOCTYPE html public>\n\n";
```

```
echo "<p>現在は", date( '%H時%i分%s秒です' ), "</p>\n";
```


こういうノリで
Webアプリがサクっと
書ける素朴な言語

PHPはこんな言語

- 汎用プログラミング言語（Webだけではない）
- 昨今ではHTMLにそのまま埋め込むよりもクラス定義が主体
- `<?php` タグの内側のシンタックスはCっぽい
 - C言語っぽい `{}` を使った制御構造がある
 - 式文が `;` で終端する

PHPとクロージャ

- 基本的にレキシカルスコープです

PHPとクロージャ

- 基本的にレキシカルスコープです
- `function ($a, $b) { return $a + $b; }` のような無名関数

PHPとクロージャ

- 基本的にレキシカルスコープです
- `function ($a, $b) { return $a + $b; }` のような無名関数
- 外側の変数をキャプチャするには明示が必要
`function ($a, $b) use ($c) { return $a + $b + $c; }`

PHPとクロージャ

- 基本的にレキシカルスコープです
- `function ($a, $b) { return $a + $b; }` のような無名関数
 - 外側の変数をキャプチャするには明示が必要
`function ($a, $b) use ($c) { return $a + $b + $c; }`
- PHP 7.4からは `fn($a, $b) => $a + $b + $c;`
 - 明示しなくても キャプチャできて便利

これだけ知れば
おもしろいこと
いろいろできそう

ところで

世の中にはいろんな
制約を課した
プログラミングがある

セミコロなしで
Javaを書くとか

Pythonワンプライナー とか

\$なしでPHPを
書くとか

<https://tadsan.fanbox.cc/posts/329540>

\$-less PHP 編集

2019年3月27日 01:15 ・ 全体公開

縛りプレイのコーディングとして、セミコロンレスJavaやPythonワンライナーが有名だがPHPでそれに匹敵する難易度の超絶技巧コーディングとして\$なしでPHPを書くことを提唱したい。

.....と思ったが、やってみると実際はそれほどの難易度ではなかった。

<https://gist.github.com/zonuxe/6da5ad0aa5a67acd310794ed79df9c39>

```
<?php
const FB = ['FizzBuzz', null, null, 'Fizz', null, 'Buzz', 'Fizz',
            null, null, 'Fizz', 'Buzz', null, 'Fizz', null, null];

call_user_func(new class {
    function __invoke()
    {
        if (func_get_arg(0) - 1 ) call_user_func(__METHOD__, func_get_arg(0) - 1);
        echo FB[func_get_arg(0) % 15] ?? func_get_arg(0), PHP_EOL;
    }
}, 100);
```


これはこれで
おもしろい

(再掲)PHPはこんな言語

- 汎用プログラミング言語（Webだけではない）
- 昨今ではHTMLにそのまま埋め込むよりもクラス定義が主体
- `<?php` タグの内側のシンタックスはCっぽい
 - C言語っぽい `{ }` を使った制御構造がある
 - 式文が `;` で終端する

(再掲)PHPはこんな言語

- 汎用プログラミング言語（Webだけではない）
- 昨今ではHTMLにそのまま埋め込むよりもクラス定義が主体
- <?php タグの内側のシンタックスはCっぽい
 - C言語っぽい **{ }** を使った制御構造がある
 - 式文が **;** で終端する

こいつら目障り
じゃないですか？

セミコロナレス
Javaでは

<https://horosora.hatenablog.com/entry/2018/08/03/162147>より引用

horolog

終わるまでは終わらないよ

2018-08-03

セミコロンレス Java をちょっとだけやってみた

Java

Java はセミコロンがなくても書ける。

ということで、セミコロンレス Java をちょっとだけやってみる。

Hello world

まずは Hello world を出力してみる。

戻り値が void 以外のメソッド呼び出しは、以下のように if 文の条件式部分で呼び出せばセミコロンがいらなくなる。

```
public class Main {  
    public static void main(String[] args) {  
        if (System.out.printf("%s", "Hello world") == null) {}  
    }  
}
```

; だけじゃなくて
{ も縛りたくない？

今回の本題

(再掲)PHPはHTMLに書く言語

```
<!DOCTYPE html public>
```

```
<p>現在は<?= date( '%H時%i分%s秒です' ) ?></p>
```

?> 直前は
;を削れる

すべての処理を
<?= に押し込む

できれば一個の
<?= に押し込む

PHPは式指向
プログラミングでできる
下地はいろいろある

PHPの特徴

- スクリプト言語にありがちな定義せずに代入したら使える変数
 - 代入は式！
- C言語とほぼ同じ制御構造なので if や while は {} または ; 必須
 - 分岐は条件演算子 `$cond ? $then : $else` でどうにか
 - ループは `array_map()`, `array_reduce()` でどうにか

これだけでできれば
チューリング完全
であることは自明

FizzBuzzを書いてみよう

1から100を改行区切りで出力

```
<?= implode(range(1, 100), "\n") ?>
```


FizzBuzzを書いてみよう

1から100をFizzBuzzに変換して改行区切りで出力

```
<?= implode(array_map(  
fn(int $n): string => [[$n, 'Buzz'],  
    ['Fizz', 'FizzBuzz']] [$n%3==0] [$n%5==0],  
range(1, 100), "\n") ?>
```

こういうノリで
map/reduceに
変形できれば
任意の処理が可能

パイプライン演算子が
欲しくなってくるぜ
(Lispならスレッドマクロがあるのに)

これはLispですね

そう思ったことが
私にもありました

ある日のこと...

<https://qiita.com/perpouh/items/807775de16986a956d1c> より引用



@perpouh

投稿日 2019年08月01日 更新日 2019年08月06日

スターリンソートwithPHP

PHP, ネタ, スターリンソート

まだPHPで書いてる人が見当たらなかったの。居たらごめんなさい。

```
<?php
$array = [1,3,4,2,3,7,3,9];
print_r(stalin_sort($array));

function stalin_sort($arr){
    $ret[] = $max = $arr[0];
    foreach($arr as $el){
        if($el <= $max) continue;
        $ret[] = $max = $el;
    }
    return $ret;
}
```

本当は練習がてらアロー関数でやろうと思ってたけど、端末に入ってたPHPが7.1だったのでやめた。
粛清というか、どちらかというと消えゆく関数の中に取り残して見捨てるみたいなコードになりました。

<https://qiita.com/perpouh/items/807775de16986a956d1c> より引用

本当は練習がてらアロー関数でやろうと思ってたけど、端末に入ってたPHPが7.1だったのでやめた。
粛清というか、どちらかというと消えゆく関数の中に取り残して見捨てるみたいなコードになりました。

これはやばい

<https://qiita.com/perpouh/items/807775de16986a956d1c> より引用



@perpouh

投稿日 2019年08月01日 更新日 2019年08月06日

スターリンソートwithPHP

PHP, ネタ, スターリンソート

まだPHPで書いてる人が見当たらなかったの。居たらごめんなさい。

```
<?php
$array = [1,3,4,2,3,7,3,9];
print_r(stalin_sort($array));

function stalin_sort($arr){
    $ret[] = $max = $arr[0];
    foreach($arr as $el){
        if($el <= $max) continue;
        $ret[] = $max = $el;
    }
    return $ret;
}
```

文を多用してるので
fn() に置き換え困難

本当は練習がてらアロー関数でやろうと思ってたけど、端末に入ってたPHPが7.1だったのでやめた。
粛清というか、どちらかというと消えゆく関数の中に取り残して見捨てるみたいなコードになりました。

この路線の限界

array_map/
array_reduceは
あくまで配列処理

無限ループできない

旧式のfunctionでは自己再帰可能

変数を参照として
キャプチャしないと
いけない

```
<?php
```

```
$fib = function (int $n) use (&$fib) {  
    return ($n < 2)  
        ? 1  
        : $fib($n - 1) + $fib($n - 2);  
};
```

アロー関数(fn)では自己再帰不能

クローージャ作成時には
クローージャ自体は作られてない
ので束縛できない

```
<?php
```

```
$fib = fn (int $n) =>  
    ($n < 2)  
    ? 1  
    : $fib($n - 1) + $fib($n - 2);
```

じゃあどうすればいい

そうですね

不動点演算子

いわゆる
Yコンビネータ
(Zコンビネータ)

Zコンビネータ

```
<?php

$fix = fn (callable $f) => call_user_func(
    (fn (callable $x) => $f(fn (...$as) => ($x($x)(...$as))))),
    (fn (callable $x) => $f(fn (...$as) => ($x($x)(...$as))))
);
```

これで勝てる

可読性重視版

```
<?= [  
  $fix = fn(callable $f) => call_user_func(  
    (fn(callable $x) => $f(fn(...$as) => ($x($x)(...$as)))),  
    (fn(callable $x) => $f(fn(...$as) => ($x($x)(...$as))))  
  ),  
  $stalin = $fix(  
    fn ($f) => fn($x = null, ...$zs) => (  
      $x !==null && $zs ? (($x <= $zs[0]) ? [$x, ...$f(...$zs)]  
        : $f($x, ...array_splice($zs, 1)))  
      : func_get_args()),  
  
    $input = [1, 2, 1, 1, 4, 3, 9],  
    json_encode($stalin(...$input)),  
  ]  
][3] ?>
```

壓縮版

```
<?= json_encode((fn(callable $f) => call_user_func(
    (fn(callable $x) => $f(fn(...$as) => ($x($x)(...$as)))),
    (fn(callable $x) => $f(fn(...$as) => ($x($x)(...$as))))
)))(fn ($f) => fn($x = null, ...$zs) => (
    $x !==null && $zs ? (($x <= $zs[0]) ? [$x, ...$f(...$zs)]
                                : $f($x, ...array_splice($zs, 1)))
    : func_get_args())
)(...[1, 2, 1, 1, 4, 3, 9])) ?>
```

完

まとめ

PHPはLispじゃない
式指向プログラミング
は大変

エクストリーム
スポーツを楽しもう

ご静聴ありがとうございました

あとPHPでもかなり
実用的な静的型検査が
できるようになってる

関連記事

- PHP式プログラミング入門（原稿版）
- PHPの無名関数式とは何か、そしてPHP7.4のアロー関数にそなえよう
- Functoolsを作った
 - Functoolsで過剰にスマートなPHPを書こう（無名再帰篇）